CARTFORGE JENKINS PROJECT

Building a Jenkins Continuous Integration Environment for CartForge Project

Project Report

Prepared by
Vrunda Koli

DevOps Engineering Course – IT Vedant
2026

1. Project Overview

CartForge Jenkins Project demonstrates the implementation of a Continuous Integration (CI) environment using Jenkins, GitHub, an Ubuntu-based AWS EC2 controller, and a Jenkins build agent. The project automates source-code checkout, dependency installation, application build, testing, packaging, and artifact delivery.

2. Project Objectives

- Install and configure Jenkins on an Ubuntu EC2 instance.
- Integrate Jenkins with a GitHub repository.
- Create and execute a Jenkins Freestyle project.
- Create and connect a permanent Jenkins agent.
- Build a Jenkins Declarative Pipeline with multiple stages.
- Configure a GitHub webhook for automatic pipeline triggering.
- Archive the generated application artifact in Jenkins.
- Monitor pipeline execution and document results.
- Maintain installation and backup scripts and project documentation.

3. Technologies Used

- AWS EC2 – cloud virtual machines for the Jenkins controller and agent.
- Ubuntu Linux – operating system used for the Jenkins environment.
- Jenkins – Continuous Integration and automation server.
- Git and GitHub – source-code version control and remote repository.
- Node.js and npm – application runtime, dependency management, build, and testing.
- Bash – automation and backup scripts.
- Groovy/Jenkins Declarative Pipeline – pipeline definition.
- tar/gzip – application packaging.

4. Application Description

CartForge is a small Node.js application created for demonstrating Jenkins CI automation. The application entry point is app.js and the project configuration is stored in package.json. The application prints successful startup and Jenkins CI messages.

The package.json scripts used by the pipeline are:

- npm install – installs project dependencies.
- npm run build – creates the dist directory and copies app.js into it.
- npm test – runs the project's automated test command.

5. System Architecture

The architecture consists of a GitHub repository connected to a Jenkins controller running on AWS EC2. The Jenkins controller manages a permanent build agent named Cartforge-Agent. A GitHub webhook notifies Jenkins when code is pushed. Jenkins then executes the pipeline on the agent and archives the generated artifact.

Workflow:

- Developer pushes source-code changes to GitHub.
- GitHub sends a webhook request to Jenkins.
- Jenkins starts the CartForge Pipeline.
- The pipeline runs on Cartforge-Agent.
- Source code is checked out.
- Dependencies are installed.
- The application is built and tested.
- The application is packaged as cartforge-application.tar.gz.
- Jenkins archives the artifact.

6. Jenkins Installation and Configuration

Jenkins was installed on an Ubuntu EC2 instance. Java was installed as a prerequisite, the Jenkins package repository was configured, and the Jenkins service was started and enabled. The Jenkins web dashboard was then accessed to complete the initial setup and create the administrator account.

Git, Node.js, and npm were also installed and verified on the Jenkins environment so that source checkout and Node.js application commands could be executed.

7. GitHub and Freestyle Job

The CartForge GitHub repository was connected to Jenkins. A Freestyle project named CartForge-Freestyle was created for the Continuous Integration setup. The job checked out the main branch, installed dependencies, ran tests, and built the application.

The successful Freestyle build demonstrated that Jenkins could retrieve the source code from GitHub and execute the required Node.js commands.

8. Jenkins Agent Configuration

A separate Ubuntu EC2 instance was configured as a Jenkins permanent agent named Cartforge-Agent. Java was installed on the agent and SSH authentication was configured between the Jenkins controller and agent. The agent was connected successfully and was used to execute Jenkins builds.

Using a separate agent demonstrates Jenkins distributed build execution and keeps build workloads separate from the controller.

9. Jenkins Backup

A Jenkins backup script was created and stored under the scripts directory. The script creates a timestamped compressed backup of the Jenkins home directory under /var/backups/jenkins.

The backup script uses the following approach:

- Generate a timestamp using the date command.
- Create a compressed tar.gz archive of /var/lib/jenkins.
- Store the archive in /var/backups/jenkins.
- Display the generated backup files.

10. Jenkins Pipeline

A Declarative Jenkins Pipeline was created in Jenkinsfile. The pipeline automates the complete CI workflow.

Stage	Purpose	Command/Action
Clone Source Code	Retrieve the main branch from GitHub	Git checkout
Install Dependencies	Install Node.js dependencies	npm install
Build Application	Build the application	npm run build
Run Tests	Execute automated tests	npm test
Package Application	Create the deployable archive	tar -czf cartforge-application.tar.gz app.js package.json dist/
Deliver Artifact	Store the artifact in Jenkins	archiveArtifacts

11. GitHub Webhook and Automatic CI

A GitHub webhook was configured to send push-event notifications to Jenkins. The Jenkins Pipeline was configured with the GitHub hook trigger for GITScm polling option. A test change was pushed to the main branch and GitHub reported a successful webhook delivery.

The push automatically triggered Jenkins Build #2. This confirmed the complete GitHub-to-Jenkins Continuous Integration workflow.

12. Pipeline Build Result

The verified Build #2 produced the following results:

- Build number: #2
- Status: SUCCESS
- Execution time: 12 seconds
- Trigger: GitHub push
- Commit: d00b83a
- Branch: main
- Jenkins agent: Cartforge-Agent
- Failed stages: None
- Artifact: cartforge-application.tar.gz

The console output confirmed successful source checkout, dependency installation, application build, testing, packaging, artifact archiving, and completion of the pipeline.

13. Pipeline Monitoring and Optimization

Pipeline monitoring was performed using Jenkins build history and console output. Build #2 completed successfully in 12 seconds with no failed stages. The generated artifact was archived successfully.

Suggested improvements include:

- Add more application test cases.
- Add automated code-quality checks.
- Add notifications for failed builds.

- Add additional security checks.
- Improve pipeline performance as the application grows.

14. Project Repository Structure

The project repository is organized to separate application files, Jenkins automation scripts, screenshots, and documentation.

```
CartForge-Jenkins-Project/
├── README.md
├── Jenkinsfile
├── pipeline-report.txt
├── app.js
├── package.json
├── package-lock.json
├── dist/
├── scripts/
│   ├── install-jenkins.sh
│   └── backup.sh
├── screenshots/
│   ├── jenkins-dashboard.png
│   ├── freestyle-job.png
│   ├── pipeline-stage-view.png
│   ├── agent-online.png
│   └── webhook-trigger.png
└── documentation/
    └── Project_Report.pdf
```

15. Challenges Faced

- Configuring Jenkins and its initial environment on AWS EC2.
- Connecting Jenkins with GitHub and verifying source checkout.
- Configuring a permanent Jenkins agent and SSH authentication.
- Correctly configuring the GitHub webhook URL.
- Verifying that a GitHub push automatically triggered the pipeline.
- Ensuring the generated artifact was successfully archived.

16. Learning Outcomes

- Understood the basic architecture of Jenkins Continuous Integration.
- Learned how Jenkins interacts with GitHub repositories.
- Learned to create Freestyle and Pipeline jobs.
- Learned how Jenkins agents execute builds.
- Learned how webhooks provide automatic CI triggers.
- Practiced Bash scripting for installation and backups.
- Learned how to package and archive build artifacts.
- Gained practical experience with AWS EC2 and Linux administration.

17. Conclusion

The CartForge Jenkins Project successfully implemented a Continuous Integration environment using Jenkins, GitHub, AWS EC2, and a dedicated Jenkins agent. The final workflow automatically responds to GitHub changes, checks out the latest source code, installs dependencies, builds and tests the application, packages it, and archives the resulting artifact. The successful GitHub-triggered Build #2 demonstrates that the CI workflow is functioning as intended.

18. Evidence and Screenshots

The following screenshots are intended to provide evidence of the project implementation and should be placed in the screenshots directory:

- jenkins-dashboard.png – Jenkins dashboard.
- freestyle-job.png – successful Freestyle job.
- pipeline-stage-view.png – Jenkins Pipeline stage view.
- agent-online.png – Cartforge-Agent connected and online.
- webhook-trigger.png – successful GitHub webhook delivery.